

I. Преговор

1. Линејни алгоритми

- изчисление на лицето на триъгълник по формулата на Херон:

$$S = \sqrt{p(p-a)(p-b)(p-c)},$$

където:

- p - полупериметър на триъгълника
- a, b, c - страни на триъгълника

- размяна на стойностите на две променливи (цели числа) чрез умножение
- изчисление на периметъра на триъгълник чрез синусовата теорема:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

където:

- a, b, c - страни на триъгълника
- A, B, C - ъгли на триъгълника
 - Въвеждат се два ъгъла и прилежаща страна

2. Разклонени алгоритми

- намиране на най-голямото от три числа
- определение дали дадена година е високосна

3. Циклични алгоритми

- изчисление на факториел - $n! = 1.2.3...n$, където n е цяло число ≥ 0

4. Едномерни масиви

- обработка на масив:
 - най-голям елемент
 - средноаритметично
 - брой нечетни числа
 - наличност на конкретно отрицателно число

5. Задачи

- Напишете програма за намиране на сумата на цифрите на естествено (цяло положително) трицифрено число.
- Напишете програма за размяна на стойностите на две променливи (цели числа) чрез събиране.
- Напишете програма за изчисление на първото $x_n > 100$ за редицата $x_{n+1} = 2x_n + 10$, където $x_0 = 2$, а $n = 0, 1, \dots$
- Напишете програма за преброяване на всички двойки съседни елементи с различни знаци.
- Напишете програма за намиране на произведението на всички елементи на едномерен масив, участващи в двойки, чиито суми са ≤ 120 .
- Напишете програма за намиране на броя на площадките (площадка - два или повече съседни еднакви елементи) на едномерен масив.

II. Матрици (двумерни масиви) и сортировки

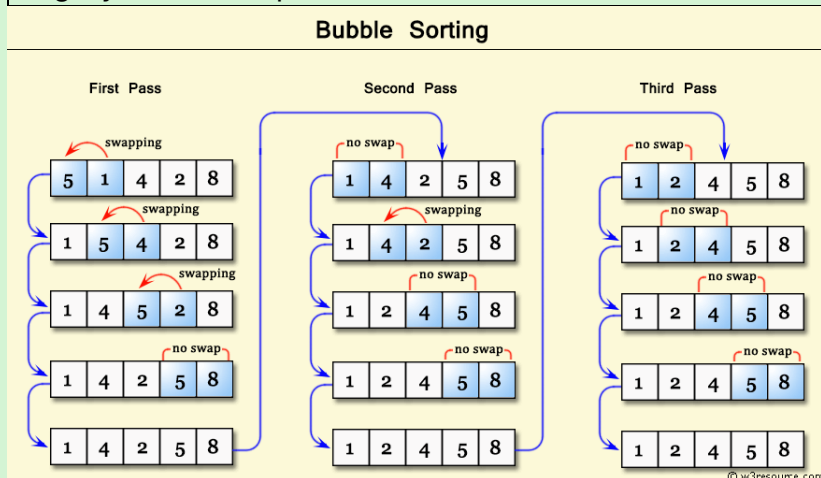
1. Матрици

- Въвеждане и извеждане на елементи
- запазване на всички елементи в определен интервал в едномерен масив
- обработка на матрица:
 - нечетните елементи над главния диагонал стават четни (инкрементация)
 - четните елементи под главния диагонал стават нечетни (декрементация)
 - при всяка промяна съгласно горните две условия, елементът от главния диагонал на същия ред се увеличава с 1
- извличане на елемент по единичен индекс

2. Сортировки

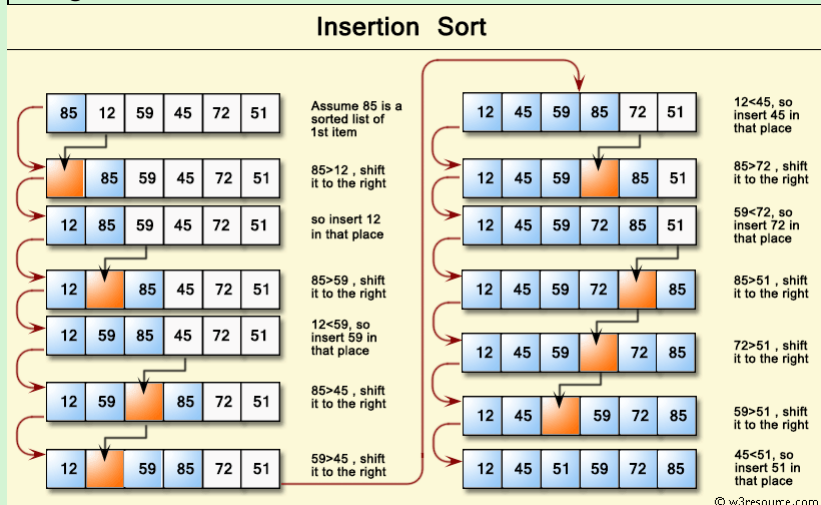
- метод на мехурчето - Bubble Sort

Подобно на изплуването на мехурчета на повърхността, елементите „изплуват“ към края на масива.



- Вмъкване - Insertion Sort

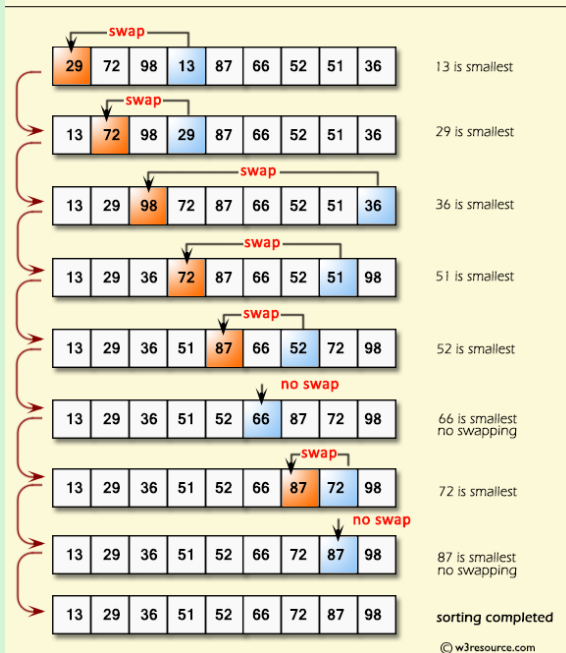
Основната идея е поставянето на несортиран елемент на мястото му във всяка итерация. Аналогично на подреждането на карти, се приема, че първата е „сортирана“. След теглене на нова карта, ако е по-голяма от старата, се слага отдясно, иначе отляво. По същия начин се постъпва с всички останали карти.



- избор (селекция) - Selection Sort

Във всяка итерация се избира най-малкият елемент в несортиран масив и се поставя в началото му.

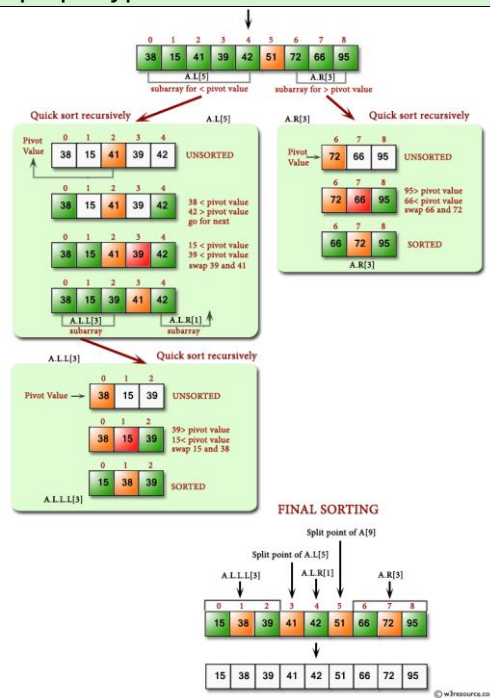
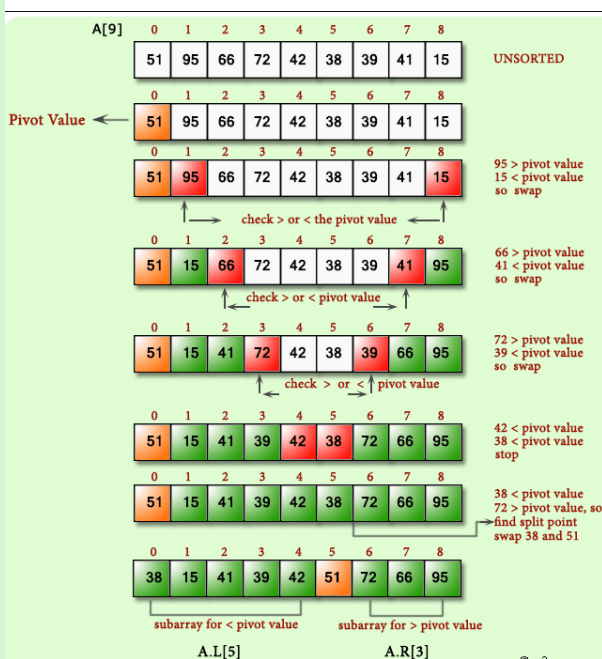
Selection Sort



• бърза - Quick Sort

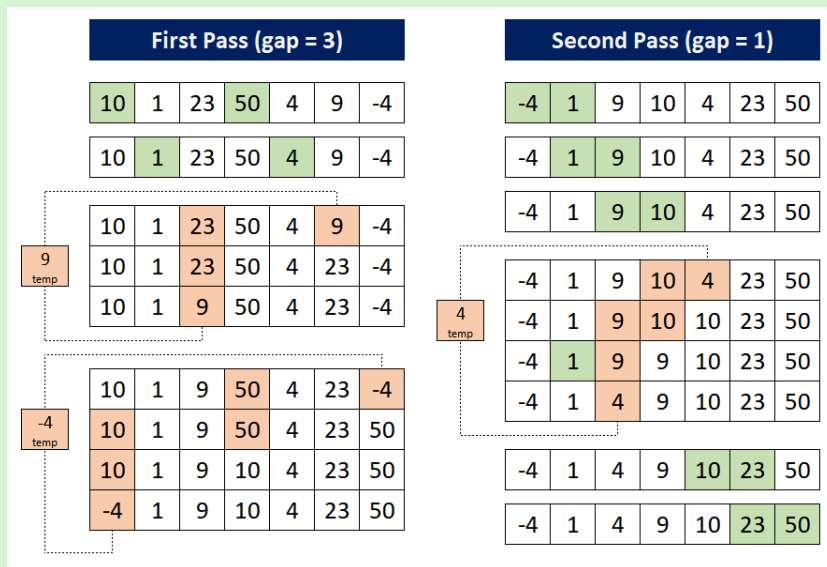
Основана на метода „разделяй и владей“ - избира се опорна точка (pivot), на базата на която по-малките елементи се местят отляво, а по-големите отясно. След това процесът се повтаря рекурсивно.

Quick Sort



• на Доналд Шел - Shell Sort

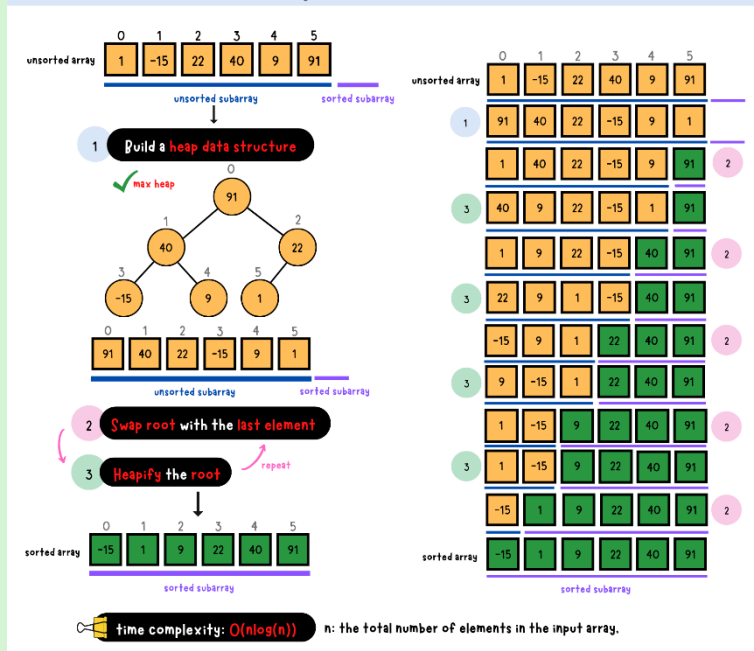
Елементите през определен интервал се сортират, като той постепенно намалява.



- пирамидална - Heap Sort

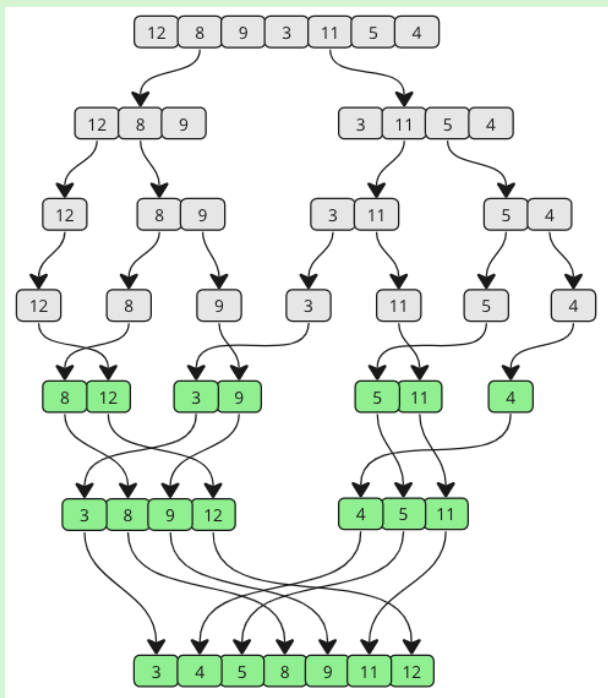
Елементите се поставят в „пирамида“ (heap) - пълно двоично дърво, по-конкретно max heap (Всеки възел е по-голям или равен на децата си). Така най-голямата стойност се „катери“ нагоре по дървото, докато не стигне корена му. След това се разменя с най-малката и се счита за подредена.

Heap Sort Algorithm



- със сливане - Merge Sort

Основана на метода „разделяй и владей“ - масивът се разделя на малки подмасиви (наполовина), които след това се сортират и постепенно се сливат.



3. Задачи

- Напишете програма, която изчислява сумата на елементите по обиколката на матрица.
- Напишете програма за обработка на матрица, която пази в едномерен масив:
 - сумата на елементите по главния диагонал
 - сумите на елементите по редове
 - броя елементи под главния диагонал, по-малки от сумите на индексите им
- Напишете програма, която сумира елементите по вторичния диагонал.
- Напишете програма за намиране на двойките елементи, симетрични на главния диагонал, в които горният елемент е по-малък от долния. В долната матрица симетричните елементи са оцветени по двойки.

a_{00}	a_{01}	a_{02}	a_{03}
a_{10}	a_{11}	a_{12}	a_{13}
a_{20}	a_{21}	a_{22}	a_{23}
a_{30}	a_{31}	a_{32}	a_{33}

III. Рекурсия

1. Рекурсия

Рекурсивната функция извиква себе си, с цел решаване на задача, която може да се раздели на еднакви подзадачи.

Задължително трябва да има базов случай за спиране, за да се избегне безкрайна рекурсия.

Обикновено такива функции са по-бавни и заемат повече памет от итеративните.

```
return 5 * factorial(4) = 120
└─ return 4 * factorial(3) = 24
    └─ return 3 * factorial(2) = 6
        └─ return 2 * factorial(1) = 2
            └─ return 1 * factorial(0) = 1
                javaTpoint.com
1 * 2 * 3 * 4 * 5 = 120
```

- изчисление на факториел (итеративно и рекурсивно)
- изчисление на НОД (най-голям общ делител) на две естествени числа
- извеждане на естествено число в обратен ред
- проверка за наличие на елемент в едномерен масив
- проверка дали редица е монотонно намаляваща

2. Задачи

- Напишете програма за преобразуване на естествено число в двоично чрез рекурсивна функция.
- Напишете програма за изчисление на x^n (n - цяло число) чрез рекурсивна функция, съгласно следните формули:
 - $n > 0$: $x^n = x * x^{n-1}$
 - $n = 0$: $x^n = 1$
 - $n < 0$: $x^n = 1/x^{-n}$
- Напишете програма за сумиране на елементите на едномерен масив чрез рекурсивна функция.
- Напишете програма за проверка за наличие на цифра в естествено число чрез рекурсивна функция.

IV. Списъци

1. Имплементация